



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Distributed Systems and Web Technologies



LUCRARE DE DISERTAȚIE

Coordonator științific:
conf. dr. ing. Cristian Nicolae BUȚINCU

Absolvent:
George-Alexandru MIRON

Iași, 2026



UNIVERSITATEA TEHNICĂ „GHEORGHE ASACHI” DIN IAȘI
FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE
Domeniul: Calculatoare și Tehnologia Informației
Programul de studii: Distributed Systems and Web Technologies



Distributed Platform for Running Services Within a Cluster

LUCRARE DE DISERTAȚIE

Coordonator științific:
conf. dr. ing. Cristian Nicolae BUTÎNCU

Absolvent:
George-Alexandru MIRON

Iași, 2026

DECLARAȚIE DE ASUMARE A AUTENTICITĂȚII LUCRĂRII DE DISERTAȚIE

Subsemnatul _____,
legitimat cu _____ seria _____ nr. _____, CNP _____
autorul lucrării _____

elaborată în vederea susținerii examenului de finalizare a studiilor de masterat,
programul de studii _____ organizat de către
Facultatea de Automatică și Calculatoare din cadrul Universității Tehnice „Gheorghe
Asachi” din Iași, sesiunea _____ a anului universitar _____,
luând în considerare conținutul Art. 34 din Codul de etică universitară al Universității
Tehnice „Gheorghe Asachi” din Iași (Manualul Procedurilor, UTI.POM.02 -
Funcționarea Comisiei de etică universitară), declar pe proprie răspundere, că această
lucrare este rezultatul propriei activități intelectuale, nu conține porțiuni plagiate, iar
sursele bibliografice au fost folosite cu respectarea legislației române (legea 8/1996) și a
convențiilor internaționale privind drepturile de autor.

Data

Semnătura

Cuprins

1	Introduction	1
1.1	Objectives	1
1.2	Methodology and Tools	1
1.3	Thesis Structure	2
2	Theoretical Background and Similar Solutions	3
2.1	Distributed Systems	3
2.1.1	Node Discovery and Heartbeats	3
2.1.2	Consensus and Leader Election	3
2.1.3	Fault Tolerance	3
2.1.4	Load Balancing	4
2.1.5	Containerization	4
2.2	Similar Solutions	4
2.2.1	Kubernetes	4
2.2.2	Docker Swarm	5
2.2.3	HashiCorp Nomad	5
2.3	Summary of Comparison	5
2.4	Expected Characteristics	6
2.5	Conclusions	6
3	Proposed Solution	7
3.1	Technology Choices	7
3.2	Component Overview	7
3.2.1	Master Node	8
3.2.2	Worker Node	9
3.2.3	Communication	9
3.3	Domain Model	9
3.4	Heartbeat and Node Discovery	10
3.4.1	Auto-Join	10
3.4.2	Heartbeat Loop	10
3.4.3	Node Removal	11
3.5	Fault Tolerance – Detection and Recovery	11
3.5.1	Failure Detection	12
3.5.2	Container Recovery	12
3.5.3	Split-Brain Prevention	12
3.6	Worker Node – State Machine	13
3.7	Implementation	14
4	Testarea aplicației și rezultate experimentale	15
4.1	Nivel 1	15
4.1.1	Nivel 2	16

4.1.1.1	Nivel 3	16
5	Concluzii	21
5.1	Direcții viitoare de dezvoltare	21
5.2	Lecții învățate pe parcursul dezvoltării proiectului de diplomă	22
	Bibliografie	23
	Anexe	25
1	Componente Software	25
2	Componente Hardware	26
3	Codul funcției xyz()	27
4	Tabel lung pe mai multe pagini	29
5	Tabel simplu	31
6	Cod în python	32
7	Cod în Kotlin (mai lung de o pagină)	33
8	Cod în XML (preluat din fisier)	35

Distributed Platform for Running Services Within a Cluster

George-Alexandru MIRON

Rezumat

Proiectul de diplomă/lucrarea de disertație va conține un scurt rezumat (abstract – termenul în engleză) de maximum o pagină, care expune ideile principale ale tezei, contribuția autorului la realizarea temei propuse precum și concluziile obținute. Formulați ideile principale ale lucrării în propoziții sau fraze cât mai simple, fără amănunte nesemnificative. Încercați să atingeți următoarele idei:

- contextul și motivația alegerii temei;
- obiectivele principale;
- metodologia utilizată;
- soluția propusă;
- rezultatele esențiale și principalele concluzii.

Rezumatul ar trebui să fie între 150-250 de cuvinte și să fie urmat de cuvinte-cheie (*keywords*, 5-7 termeni relevanți).

Recomandare neoficială: Rezumatul se compune, de obicei, după ce ați terminat redactarea lucrării de licență/disertație, după finalizarea capitolului Concluzii

Capitolul 1. Introduction

Running services reliably across multiple machines is a common challenge in modern software systems. When an application grows beyond what a single server can handle, it needs to be distributed across a cluster of nodes. This brings new problems: how do nodes discover each other, how are resources allocated, what happens when a node fails, and how do we keep the system secure.

Large-scale platforms like Kubernetes solve these problems, but they come with significant complexity. For many use cases, a simpler solution is enough. This thesis proposes a distributed platform that addresses the core challenges of running services in a cluster, while keeping the architecture straightforward and easy to understand.

1.1. Objectives

The main objective of this thesis is to design and implement a distributed platform for running containerized services within a cluster. The platform addresses the following specific objectives:

- **Automatic node discovery** – worker nodes are discovered automatically by the master when they send their first heartbeat via UDP multicast. No explicit join or leave mechanism is needed.
- **Resource pools** – nodes are grouped into pools based on their hardware resources (CPU, RAM). Services can be deployed to a specific pool, ensuring they run on appropriate hardware.
- **Load balancing** – when deploying a service, the platform selects the best node using a pluggable load balancing strategy (Least Connections or Weighted).
- **Fault tolerance** – the platform detects node failures through heartbeat monitoring and automatically restarts affected containers on healthy nodes. A Raft consensus protocol ensures master node availability.
- **Security** – user authentication is handled through JWT tokens, and a role-based access control system (Admin, Writer, Reader) restricts what each user can do.
- **Audit** – all important actions are logged in an append-only audit log that can be queried through the API.

1.2. Methodology and Tools

The platform is implemented in Go, chosen for its built-in concurrency support (goroutines), efficient networking, and suitability for systems programming. The main tools and libraries used are:

- **Go** – the programming language for both master and worker components
- **Docker SDK** – for managing containers on worker nodes
- **HashiCorp Raft** – for leader election and state replication between master nodes
- **UDP multicast** – for heartbeat communication between workers and master
- **JWT (JSON Web Tokens)** – for user authentication

1.3. Thesis Structure

The thesis is organized into five chapters:

- **Chapter 1 – Introduction:** presents the context, objectives, and methodology.
- **Chapter 2 – Theoretical Background and Similar Solutions:** covers distributed systems concepts and compares existing platforms like Kubernetes and Docker Swarm.
- **Chapter 3 – Proposed Solution:** describes the architecture of the platform in detail, including component design, communication protocols, state machines, and security mechanisms.
- **Chapter 4 – Testing and Experimental Results:** presents the testing methodology and results obtained from running the platform.
- **Chapter 5 – Conclusions:** summarizes the results and discusses future improvements.

Capitolul 2. Theoretical Background and Similar Solutions

This chapter presents the key concepts behind distributed systems and reviews existing platforms that solve similar problems. Understanding these concepts is important for explaining the design decisions made in the proposed solution.

2.1. Distributed Systems

A distributed system is a collection of independent computers that appear to the user as a single system [1]. The main challenges in distributed systems are:

- **Partial failure** – some nodes can fail while others continue to work. The system must handle this without stopping entirely.
- **No global clock** – each node has its own clock, and there is no single source of time for the entire system.
- **Network unreliability** – messages between nodes can be delayed, lost, or delivered out of order.

2.1.1. Node Discovery and Heartbeats

In a cluster, nodes need to know about each other. There are two common approaches:

Static configuration – the list of nodes is defined in a configuration file. Simple, but does not support dynamic scaling.

Heartbeat-based discovery – each node periodically sends a small message (heartbeat) to announce that it is alive. If a node stops sending heartbeats, it is considered dead. This approach is used by systems like Consul [2] and the proposed platform.

Heartbeats can be sent via TCP (reliable, but more overhead) or UDP (lightweight, no connection setup). UDP multicast allows a single packet to reach multiple receivers at once, which is efficient when multiple master nodes need to receive heartbeats [3].

2.1.2. Consensus and Leader Election

When multiple master nodes need to agree on the state of the cluster, they use a consensus protocol. The most widely used consensus protocol in modern systems is **Raft** [4].

Raft works by electing a leader among the master nodes. The leader processes all write operations and replicates them to the followers. If the leader fails, the followers elect a new leader. Raft guarantees that:

- Only one leader exists at any time.
- All nodes agree on the same sequence of operations.
- The system continues to work as long as a majority of nodes (quorum) are alive. For 3 nodes, the quorum is 2.

2.1.3. Fault Tolerance

Fault tolerance is the ability of a system to continue operating when some of its components fail [1]. Common strategies include:

- **Replication** – running multiple copies of a service on different nodes. If one copy fails, others continue to serve requests.

- **Health monitoring** – periodically checking if nodes are alive (via heartbeats) and taking action when they are not.
- **Automatic recovery** – restarting failed services on healthy nodes without human intervention.

2.1.4. Load Balancing

Load balancing distributes work across multiple nodes to avoid overloading any single node. Common strategies include:

- **Round Robin** – requests are distributed to nodes in a circular order.
- **Least Connections** – the node with the fewest active connections is chosen.
- **Weighted** – nodes with more available resources receive more work.

The proposed platform implements Least Connections and Weighted strategies with a pluggable interface, allowing new strategies to be added without modifying existing code.

2.1.5. Containerization

Containers are lightweight, isolated environments for running applications. Unlike virtual machines, containers share the host operating system kernel, which makes them faster to start and more efficient in resource usage [5].

Docker is the most widely used container runtime. It provides an API for creating, starting, stopping, and removing containers. The proposed platform uses the Docker SDK to manage containers on worker nodes.

2.2. Similar Solutions

Several platforms exist for running containerized services in a cluster. This section compares the most relevant ones with the proposed solution.

2.2.1. Kubernetes

Kubernetes [6] is the industry standard for container orchestration. It was originally developed by Google and is now maintained by the Cloud Native Computing Foundation (CNCF).

Key features:

- Automatic scheduling and scaling of containers
- Self-healing: restarts failed containers, replaces nodes
- Service discovery and load balancing
- Configuration management and secrets
- Extensible through custom resources and operators

Kubernetes uses etcd (a distributed key-value store based on Raft) for cluster state, and kubelet agents on each node for container management.

Comparison with the proposed platform: Kubernetes is a full production platform with hundreds of features. The proposed platform focuses on the core mechanisms (heartbeats, scheduling, fault tolerance, security) and implements them from scratch, providing a clear and simple architecture that is easier to understand and extend. Kubernetes is designed for large-scale production use, while the proposed platform is designed for educational purposes and smaller deployments.

2.2.2. Docker Swarm

Docker Swarm [7] is Docker's built-in orchestration tool. It is simpler than Kubernetes and integrates directly with the Docker CLI.

Key features:

- Uses the standard Docker API
- Built-in service discovery
- Load balancing via ingress routing
- Raft consensus for manager nodes
- TLS encryption for node communication

Comparison with the proposed platform: Docker Swarm is closer in simplicity to the proposed platform. Both use Raft for consensus and Docker for container management. The main differences are that the proposed platform uses UDP multicast for heartbeats (instead of TCP), does not require TLS for internal communication (trusted network assumption), and implements auto-join via heartbeats instead of explicit join tokens.

2.2.3. HashiCorp Nomad

Nomad [8] is a workload orchestrator developed by HashiCorp. Unlike Kubernetes, it can manage not only containers but also standalone applications, Java services, and batch jobs.

Key features:

- Single binary, easy to deploy
- Multi-datacenter support
- Integrates with Consul for service discovery and Vault for secrets
- Uses Raft for consensus

Comparison with the proposed platform: Nomad shares the philosophy of simplicity. Both use Raft for consensus and a master-worker architecture. The proposed platform is more focused – it only manages Docker containers and implements all components (discovery, scheduling, security) in a single system, without external dependencies like Consul or Vault.

2.3. Summary of Comparison

Table 2.1 summarizes the key differences between the existing solutions and the proposed platform.

Tabelul 2.1. Comparison of existing solutions with the proposed platform

Feature	Kubernetes	Swarm	Nomad	Proposed
Consensus	etcd (Raft)	Raft	Raft	Raft
Node discovery	kubelet	join token	Consul	heartbeat
Heartbeat protocol	TCP	TCP	TCP	UDP multicast
Container runtime	multiple	Docker	multiple	Docker
Internal encryption	optional	TLS	TLS	none (trusted)
Complexity	very high	medium	medium	low
Load balancing	yes	yes	no (external)	yes
RBAC	yes	limited	yes	yes
Audit log	yes	no	yes	yes

2.4. Expected Characteristics

Based on the analysis of existing solutions and the identified gaps, the proposed platform should meet the following requirements:

- **Automatic node discovery** – worker nodes should be detected automatically when they start, without manual configuration or explicit join commands.
- **Failure detection within 30 seconds** – the system should detect a dead node within 30 seconds of the last heartbeat and begin container recovery immediately.
- **Container recovery without data loss** – when a node dies, all service parameters (image, environment variables, ports, command) should be preserved in the cluster state, allowing containers to be restarted identically on healthy nodes.
- **Pluggable load balancing** – the load balancing strategy should be interchangeable without modifying the rest of the code.
- **Role-based access control** – the platform should support at least three roles (Admin, Writer, Reader) with clearly separated permissions.
- **Audit trail** – all important actions should be logged in an append-only log that can be queried through the API.
- **High availability** – the master should be replicated across at least 3 nodes using Raft consensus, tolerating the failure of 1 master node.
- **Low complexity** – the platform should be simple enough to understand, deploy, and extend, unlike production platforms such as Kubernetes.

2.5. Conclusions

The existing platforms are mature and feature-rich, but they come with significant complexity. Kubernetes, in particular, has a steep learning curve and requires extensive infrastructure to run.

The proposed platform fills a gap by implementing the core distributed systems mechanisms (heartbeat-based discovery, Raft consensus, pluggable load balancing, role-based security, and audit logging) in a simple, understandable way. It is not meant to replace Kubernetes or Docker Swarm in production, but to serve as a clear implementation of the fundamental concepts, suitable for educational use and smaller deployments.

Capitolul 3. Proposed Solution

The problem addressed by this thesis is running containerized services reliably across a cluster of machines. The key challenges are: discovering nodes automatically, distributing workloads efficiently, recovering from failures without downtime, and controlling access to the platform. Existing solutions like Kubernetes solve these problems but are too complex for smaller deployments and educational use.

The proposed solution is a distributed platform built from scratch in Go that implements the core mechanisms needed to manage a cluster: heartbeat-based node discovery, resource pools, pluggable load balancing, automatic fault recovery, JWT-based security with role-based access control, and an append-only audit log. The cluster state is replicated across multiple master nodes using the Raft consensus protocol.

This chapter describes the architecture in detail. The platform has two main components: the **master node**, which manages the cluster, and the **worker nodes**, which run the containers.

3.1. Technology Choices

The following technologies were chosen for the implementation:

- **Go** – chosen as the main language for its built-in concurrency model (goroutines and channels), efficient networking libraries, single-binary compilation, and strong support for systems programming. Both the master and worker are written in Go.
- **UDP multicast** – used for heartbeats because it is lightweight (no connection setup, no handshake) and allows a single packet to reach all master nodes simultaneously. This is more efficient than TCP for periodic status messages that do not require acknowledgment.
- **Docker** – used as the container runtime on worker nodes. The Docker SDK for Go provides a simple API for creating, starting, and stopping containers programmatically.
- **HashiCorp Raft** – an open-source Go implementation of the Raft consensus protocol. Used for leader election and state replication between master nodes.
- **JWT (JSON Web Tokens)** – used for user authentication. JWTs are stateless (no server-side session storage needed) and can encode the user's role directly in the token.

3.2. Component Overview

The platform follows a master-worker architecture. The master node handles all management tasks, while worker nodes execute containers and report their status.

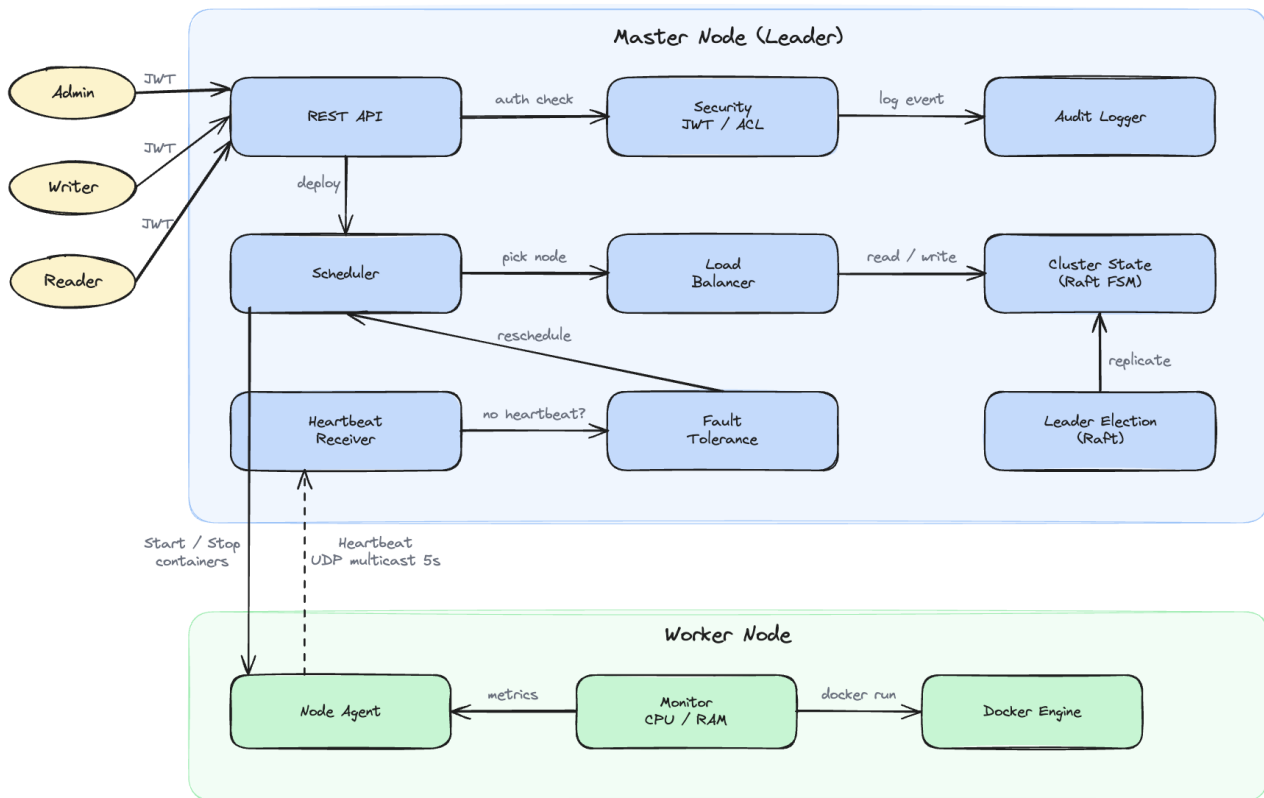


Figura 3.1. Component Overview

3.2.1. Master Node

The master node contains the following modules:

Request pipeline:

- **REST API** – the entry point for users. Exposes endpoints for managing services, nodes, and pools.
- **Security** – handles JWT authentication and ACL authorization. Every request passes through this module before being processed.
- **Audit Logger** – records all important events in an append-only log.

Scheduling pipeline:

- **Scheduler** – receives deploy requests and filters nodes in the requested pool that have enough resources. Passes the candidate list to the Load Balancer.
- **Load Balancer** – selects the best node from the candidate list. Supports two strategies: Least Connections and Weighted.
- **Cluster State (Raft FSM)** – the source of truth for the entire cluster. Stores all data about nodes, services, containers, pools, and users. Replicated across 3 master nodes via Raft.

Cluster health:

- **Heartbeat Receiver** – listens for UDP multicast packets from workers every 5 seconds.
- **Fault Tolerance** – monitors heartbeats and detects node failures. Marks nodes as SUSPECT after 15 seconds and DEAD after 30 seconds without a heartbeat.
- **Leader Election (Raft)** – manages leader election across 3 master nodes using the Raft consensus protocol.

3.2.2. Worker Node

Each worker node runs three components:

- **Monitor** – reads system metrics (CPU usage, RAM usage) from the operating system.
- **Node Agent** – the main process. Sends heartbeats every 5 seconds and listens for commands from the master (StartContainer, KillContainers).
- **Docker Engine** – the agent interacts with Docker to start and stop containers.

3.2.3. Communication

Master and worker nodes communicate through two channels:

- **Heartbeat** – UDP multicast, every 5 seconds. The worker sends its metrics and the master updates its state. Fire-and-forget, no acknowledgment needed.
- **Commands** – TCP, from master to worker. Used for StartContainer and KillContainers. Only triggered on deploy, scale, or fault recovery.

All nodes run within a trusted internal network, so no TLS encryption is used for node-to-master communication. Raft communication between master nodes is encrypted with mTLS.

3.3. Domain Model

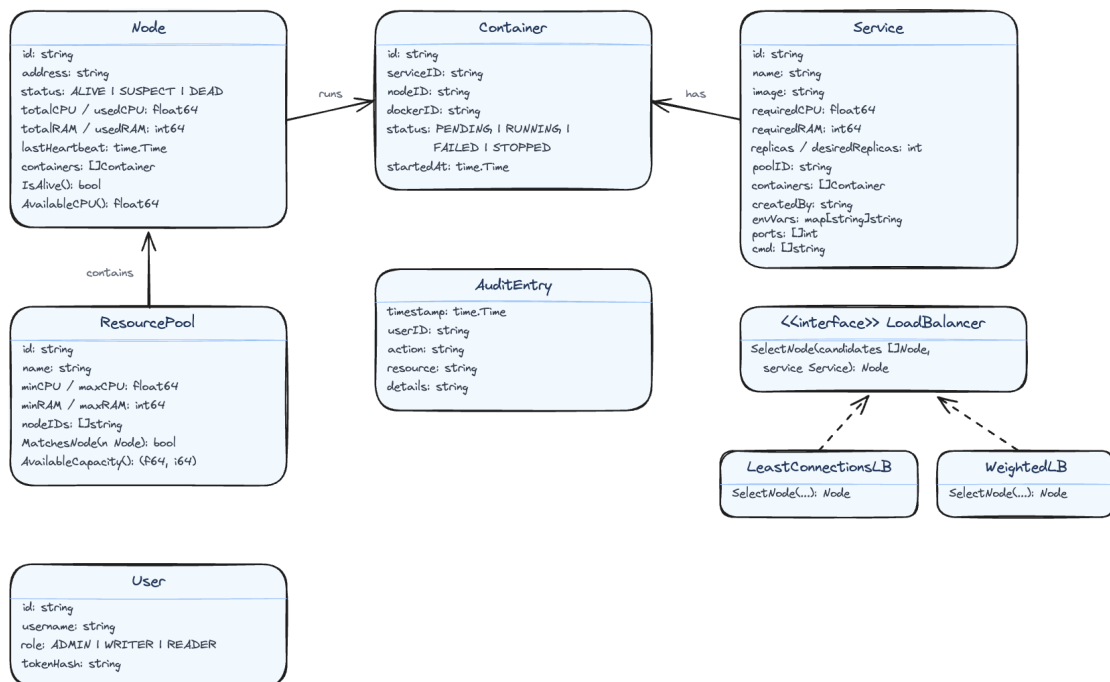


Figura 3.2. Domain Model

The platform uses the following data entities:

Node represents a worker machine in the cluster. It stores the node's address, total and used CPU/RAM, its current status (ALIVE, SUSPECT, DEAD, or REMOVED), and the timestamp of the last received heartbeat.

Service represents what the user wants to run. It contains the Docker image name, required CPU and RAM, the number of desired replicas, the target pool, and runtime parameters (environment

variables, ports, command). These parameters are saved in the cluster state so that containers can be restarted identically during fault recovery.

Container is a running instance of a service on a specific node. It tracks the Docker container ID and its status (PENDING, SCHEDULED, RUNNING, FAILED, RESCHEDULING, or STOPPED).

ResourcePool groups nodes with similar hardware. Each pool has CPU and RAM ranges. When a new node is discovered, it is automatically assigned to the matching pool based on its resources.

User has an ID, username, role (ADMIN, WRITER, or READER), and a token hash for authentication.

AuditEntry records a single event with a timestamp, user ID, action, resource, and details.

LoadBalancer is an interface with a single method: `SelectNode`. It has two implementations: `LeastConnectionsLB` (picks the node with fewest active connections) and `WeightedLB` (picks the node with most available resources).

3.4. Heartbeat and Node Discovery

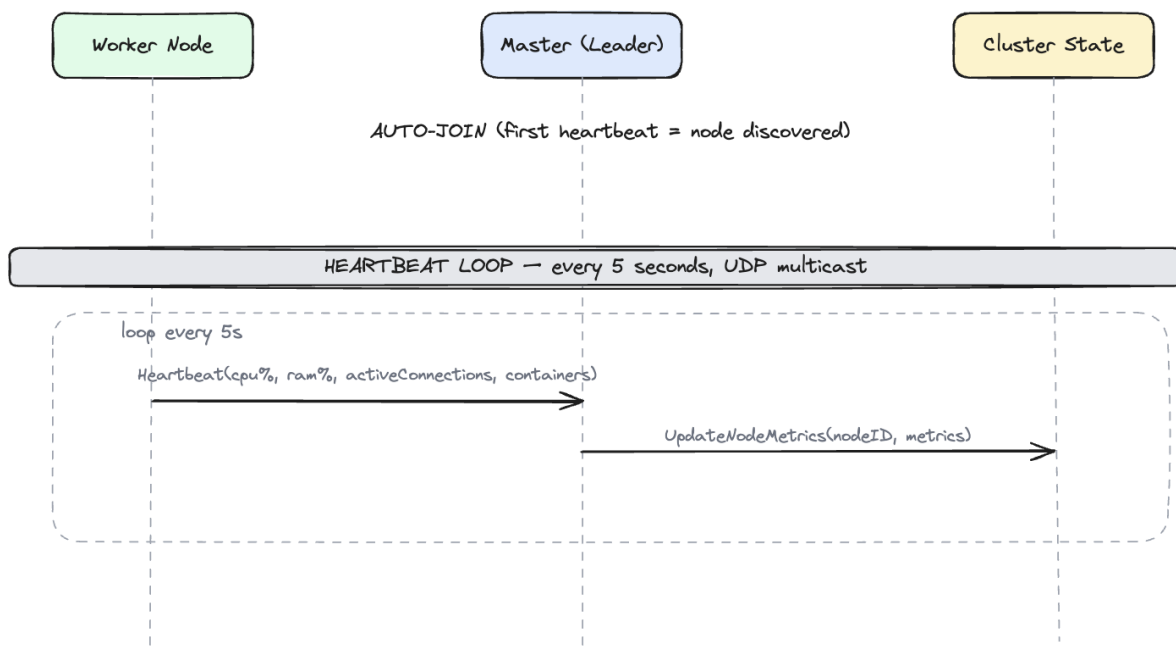


Figura 3.3. Heartbeat and Node Discovery

Node discovery is fully automatic. There is no explicit join or leave request.

3.4.1. Auto-Join

When the master receives the first heartbeat from an unknown node, it automatically:

1. Adds the node to the cluster state with status ALIVE.
2. Assigns it to the matching resource pool based on CPU and RAM reported in the heartbeat.

3.4.2. Heartbeat Loop

Every 5 seconds, each worker sends a UDP multicast packet containing:

- CPU usage (percentage)

- RAM usage (percentage)
- Number of active connections
- List of running containers

The master receives the packet and updates the node's metrics in the cluster state. No response is sent back – the heartbeat is fire-and-forget.

3.4.3. Node Removal

There is no explicit leave request. When a worker stops (crash, shutdown, or network failure), it simply stops sending heartbeats. The master detects the absence through the fault tolerance mechanism (15 seconds SUSPECT, 30 seconds DEAD) and removes the node from the cluster.

3.5. Fault Tolerance – Detection and Recovery

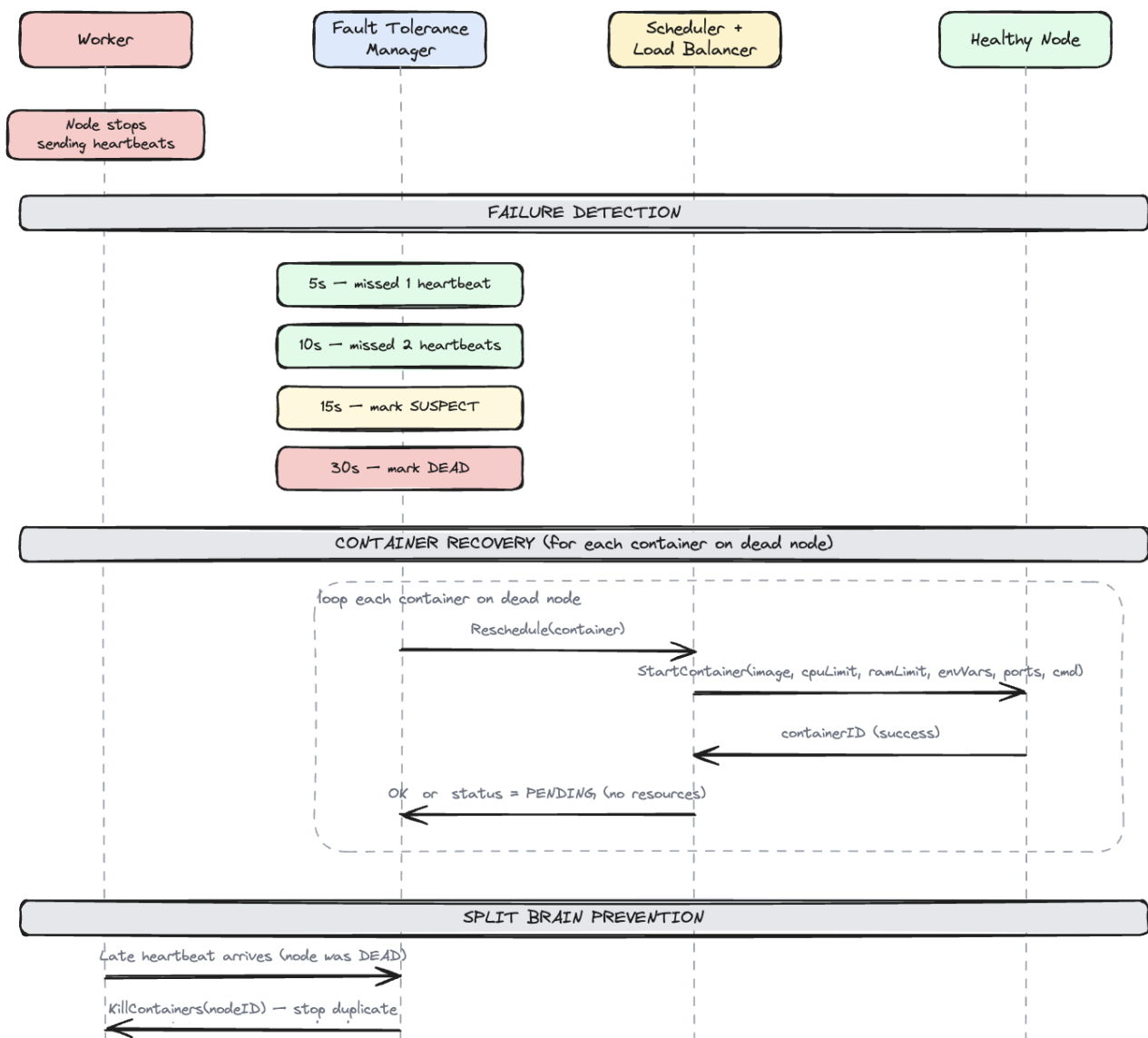


Figura 3.4. Fault Tolerance – Detection and Recovery

3.5.1. Failure Detection

The Fault Tolerance module runs a periodic check (every 5 seconds) on all nodes. For each node, it computes the time since the last heartbeat:

- **5 seconds** – 1 missed heartbeat. Could be network jitter. No action.
- **10 seconds** – 2 missed heartbeats. No action yet.
- **15 seconds** – 3 missed heartbeats. The node is marked **SUSPECT**.
- **30 seconds** – 6 missed heartbeats. The node is marked **DEAD**. Recovery begins.

3.5.2. Container Recovery

When a node is marked DEAD, its services still need to run. The system starts fresh containers on healthy nodes. For each service that had containers on the dead node:

1. The Fault Tolerance module asks the Scheduler to find a new node.
2. The Scheduler filters nodes in the matching pool with enough resources.
3. The Load Balancer picks the best node.
4. A StartContainer command is sent to the chosen node with the full service parameters (image, CPU limit, RAM limit, environment variables, ports, command).
5. If no node has enough resources, the container stays in PENDING status until resources become available.

The key point is that containers are not moved from the dead node – they are started fresh from the service parameters saved in the cluster state.

3.5.3. Split-Brain Prevention

When a node that was marked DEAD comes back online (for example, after a temporary network failure), it sends a heartbeat. The master detects this as a late heartbeat and sends a KillContainers command to the recovered node. This stops all containers on the recovered node, because those containers have already been rescheduled onto other healthy nodes. Without this mechanism, the same containers would run on both the recovered node and the new nodes, creating duplicates.

After killing the duplicate containers, the recovered node continues sending heartbeats and becomes available for new scheduling.

3.6. Worker Node – State Machine

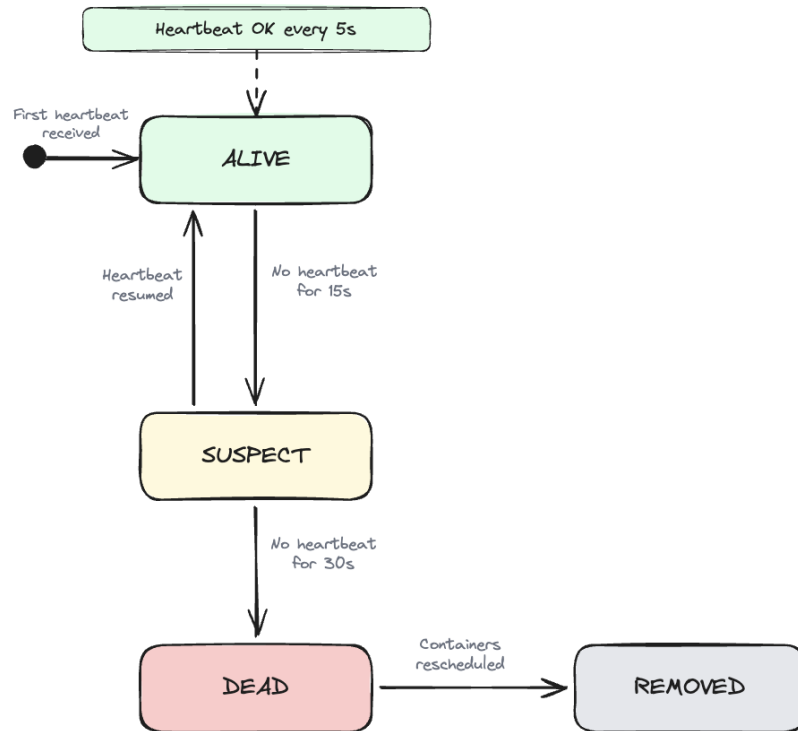


Figura 3.5. Worker Node – State Machine

A worker node can be in one of four states:

- **ALIVE** – the initial and normal state. A node enters this state when the master receives its first heartbeat. It sends heartbeats every 5 seconds, runs containers, and is available for scheduling.
- **SUSPECT** – the master has not received a heartbeat for 15 seconds. The node might be temporarily unreachable. No action is taken yet.
- **DEAD** – no heartbeat for 30 seconds. The master starts fresh containers on other nodes using the original service parameters.
- **REMOVED** – the node has been removed from the cluster state. All its containers have been rescheduled.

The transitions between states are:

- First heartbeat received → **ALIVE** (auto-join)
- Heartbeat OK every 5s → stays **ALIVE** (self-loop)
- No heartbeat for 15s → **SUSPECT**
- Heartbeat resumed → back to **ALIVE**
- No heartbeat for 30s → **DEAD**
- Containers rescheduled → **REMOVED**

3.7. Load Balancer

The Load Balancer selects the best node from a list of candidates. It is defined as a Go interface with a single method:

```
type LoadBalancer interface {
    SelectNode(candidates []*Node, service *Service) *Node
}
```

This design allows the load balancing strategy to be swapped without modifying the rest of the code. The platform provides two implementations:

LeastConnectionsLB picks the node with the fewest running containers. It iterates through the candidate list and selects the node with the smallest container count. This strategy works well when all containers have similar resource requirements.

WeightedLB picks the node with the most available resources. It calculates a score for each node based on available CPU and available RAM (converted to CPU-equivalent units), and selects the node with the highest score. This strategy is better when containers have varying resource needs.

The master can be started with either strategy using the `-lb` flag: `-lb leastconn` (default) or `-lb weighted`.

3.8. Service Deployment Flow

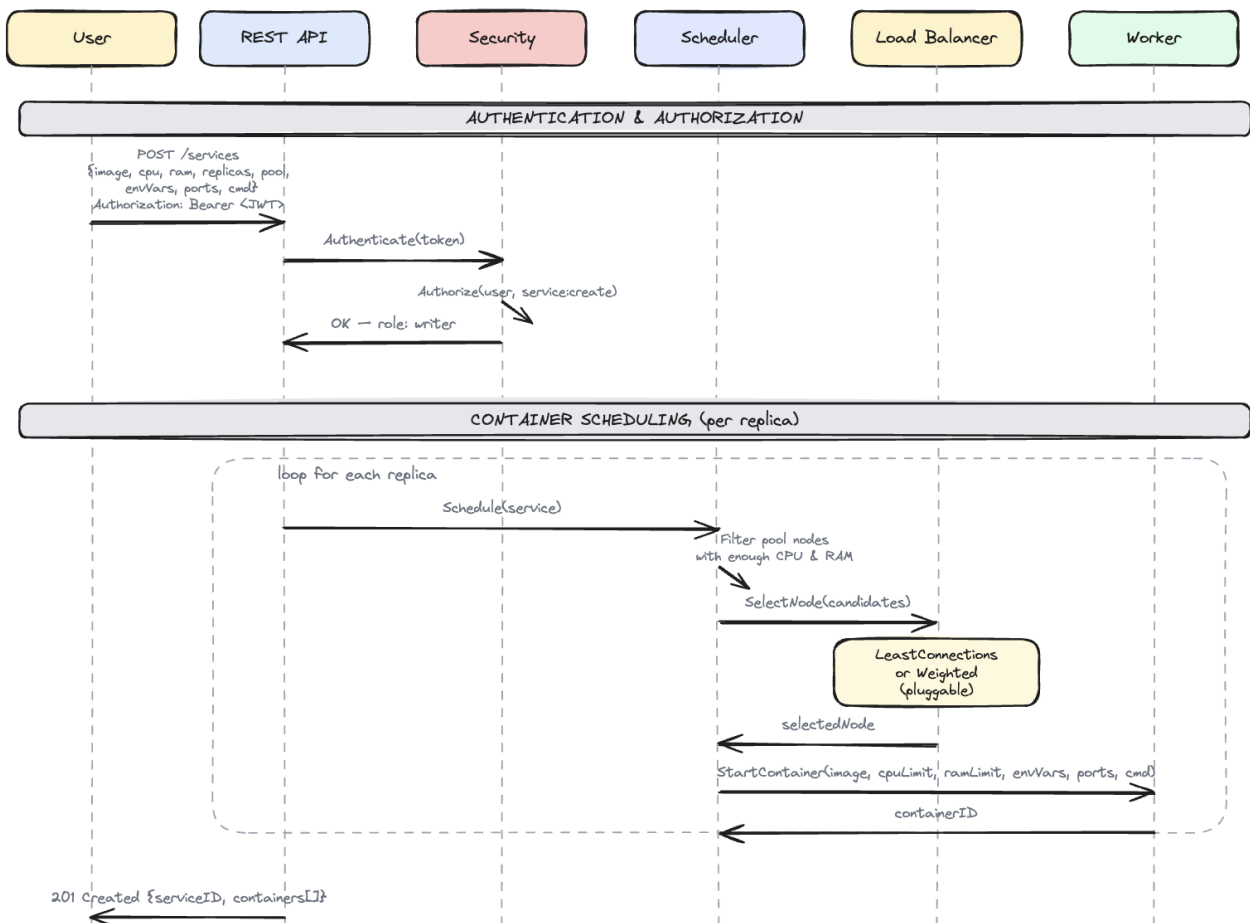


Figura 3.6. Service Deployment Flow

When a service needs to be deployed, the Scheduler and Load Balancer work together to find the best node. The Scheduler performs the following steps:

1. Retrieves all nodes in the requested resource pool.
2. Filters out nodes that are not ALIVE or do not have enough CPU and RAM for the service.
3. Passes the candidate list to the Load Balancer.
4. The Load Balancer returns the best node.
5. A `StartContainer` command is sent to the selected node via TCP with the full service parameters (image, CPU limit, RAM limit, environment variables, ports, command).
6. The worker receives the command, starts the container, and returns the container ID.

This same flow is used both for new deployments and for fault recovery (when containers need to be rescheduled after a node dies).

3.9. Agent – Worker Command Server

Each worker runs a TCP command server that listens for commands from the master. The server accepts two types of commands:

StartContainer – the master sends the Docker image, resource limits, and service parameters. The worker starts a new container and returns the container ID.

KillContainers – the master sends this command during split-brain prevention. The worker stops all running containers.

Commands are sent as JSON over TCP. The master uses the node's address (reported in heartbeats) to establish a connection, sends the command, and waits for a response. This communication is synchronous – the master waits for the worker to confirm before proceeding.

3.10. Container – State Machine

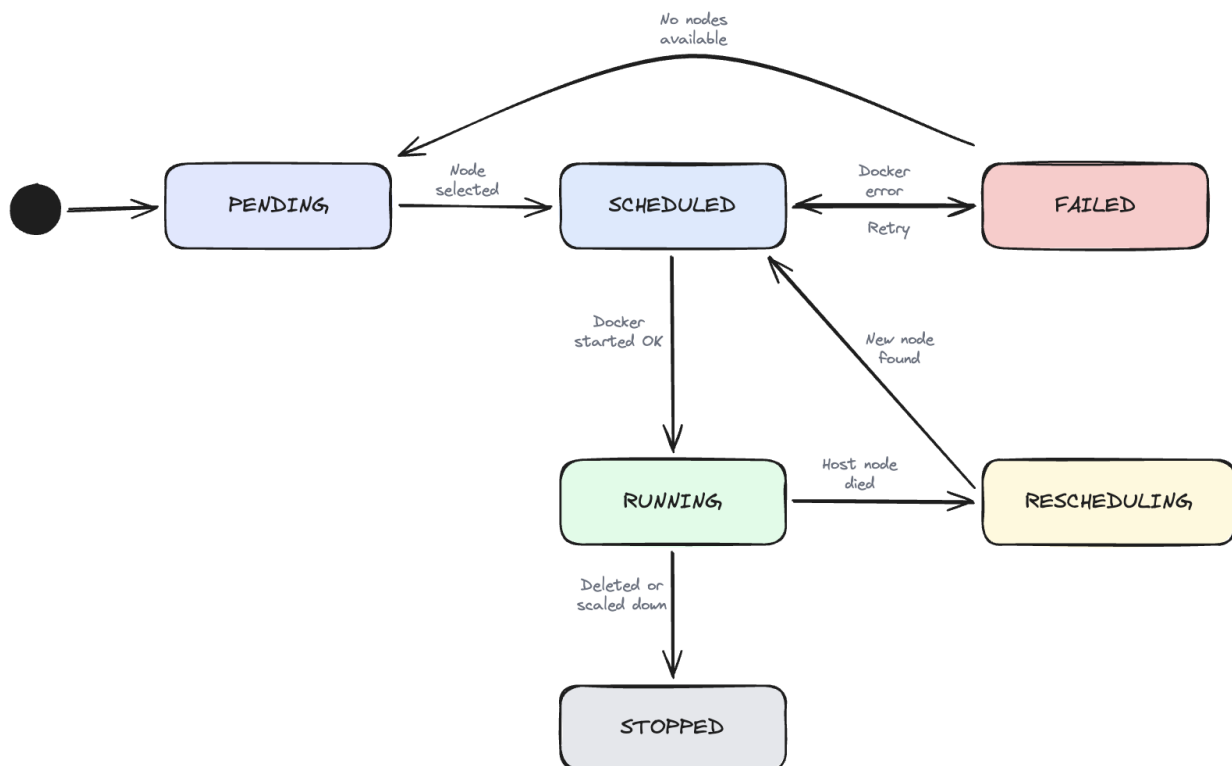


Figura 3.7. Container – State Machine

A container goes through the following states:

- **PENDING** – the container needs to run, but no node has enough resources. It waits until resources become available.
- **SCHEDULED** – a node was found and the container has been assigned. Docker startup is in progress.
- **RUNNING** – Docker started the container successfully. This is the normal operating state.
- **FAILED** – Docker returned an error (invalid image, port conflict, out of memory). The system will retry on another node.
- **RESCEDULING** – the host node died. A fresh container needs to be started on another node.
- **STOPPED** – the container was stopped intentionally (service deleted or scaled down). This is a final state.

3.11. Implementation

The platform is structured as a single Go module with two binaries: master and worker. The following packages have been implemented:

- `internal/domain` – data models (Node, Service, Container, ResourcePool, User, AuditEntry, Heartbeat)
- `internal/cluster` – in-memory cluster state with thread-safe CRUD operations, protected by `sync.RWMutex`
- `internal/heartbeat` – UDP multicast sender (worker) and receiver (master) with auto-join on first heartbeat
- `internal/fault` – fault tolerance: periodic health checks, SUSPECT/DEAD detection, KillContainers for split-brain prevention
- `internal/loadbalancer` – LoadBalancer interface with LeastConnections and Weighted implementations
- `internal/scheduler` – filters nodes by pool and resources, delegates to Load Balancer for node selection
- `internal/agent` – TCP command server (StartContainer, KillContainers), system metrics monitor (CPU, RAM), and command client for master-to-worker communication

Capitolul 4. Testarea aplicației și rezultate experimentale

(aproximativ 7 pagini)

- Punerea în funcțiune/lansarea aplicației,elemente de configurare sau instalare;
- Testarea sistemului (hardware/software);
- Aspecte legate de încărcarea procesorului, memoriei, limitări în ce privește transmisia datelor/comunicarea;
- Se prezintă datele de test/metrici/benchmarks
- Aspecte legate de fiabilitate/securitate/scalabilitate;
- Rezultate experimentale (grafice, tabele, etc.);
- Utilizarea sistemului.

4.1. Nivel 1

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue

quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus. (Figura 4.1)



Figura 4.1. Clone Trooper¹

Tabelul 4.1 prezintă un exemplu foarte simplu de tabel în $\text{\LaTeX}$. Un alt exemplu este prezentat în Tabelul A.2 din 3.

4.1.1. Nivel 2

4.1.1.1. Nivel 3

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus

¹ imagine preluată de pe un site web care nu „merită” trecut la bibliografie <https://www.pngitem.com/>

Tabelul 4.1. Exemplu de tabel simplu

Col1	Col2	Col3	Col4
1	6	87837	787
2	7	78	5415
3	545	778	7507
4	545	18744	7560
5	88	788	6344

rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui

lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel,

eleifend faucibus, vehicula eu, lacus.

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Capitolul 5. Concluzii

Concluziile lucrării (1–2 pagini) în care se regăsesc cele mai importante concluzii din lucrare, care pornind de la obiectivele propuse în introducerea lucrării vor conține opinia personală a autorului privind rezultatele obținute în lucrare, o comparație a acestora cu alte proiecte/produse similare precum și posibile direcții de dezvoltare.

- Gradul în care s-a realizat tema propusă (motivarea eventualelor obiective modificate);
- Evidențierea concisă a contribuțiilor/soluțiilor personale (dacă este cazul);
- Comparație cu alte proiecte similare;
- Posibile direcții de dezvoltare.

În acest capitol nu se introduc figuri, table sau listing-uri de cod. Pot fi în schimb referite!

Concluziile finale au un rol semnificativ în cadrul lucrării de diplomă, care trebuie pus în evidență în special în momentul susținerii acesteia în fața comisiei. Aici trebuie prezentate, în câteva pagini, cele mai importante rezultate ale muncii depuse de candidat și potențiale direcții de dezvoltare ulterioară a temei abordate. Acest capitol oferă o sinteză a realizărilor obținute, precum și avantajele și neajunsurile soluțiilor alese. Concluziile nu trebuie să aducă rezultate sau interpretări noi, ci doar să le evidențieze pe cele din conținutul lucrării.

5.1. Direcții viitoare de dezvoltare

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

5.2. Lecții învățate pe parcursul dezvoltării proiectului de diplomă

Ar fi interesant dacă acest capitol s-ar încheia cu o astfel de secțiune, în mod special la licență.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Bibliografie

- [1] A. S. Tanenbaum and M. V. Steen, *Distributed Systems: Principles and Paradigms*, 3rd ed. Pearson, 2017, ISBN: 978-1-5430-5738-6.
- [2] HashiCorp, “Consul documentation,” <https://developer.hashicorp.com/consul/docs>, 2024, Ultima accesare: Mai 2026.
- [3] W. R. Stevens, B. Fenner, and A. M. Rudoff, *UNIX Network Programming, Volume 1: The Sockets Networking API*, 3rd ed. Addison-Wesley, 2003, ISBN: 978-0-13-141155-5.
- [4] D. Ongaro and J. Ousterhout, “In Search of an Understandable Consensus Algorithm,” in *Proceedings of the 2014 USENIX Annual Technical Conference (USENIX ATC '14)*, 2014, pp. 305–319.
- [5] Docker Inc., “Docker overview,” <https://docs.docker.com/get-started/overview/>, 2024, Ultima accesare: Mai 2026.
- [6] The Kubernetes Authors, “Kubernetes documentation,” <https://kubernetes.io/docs/>, 2024, Ultima accesare: Mai 2026.
- [7] Docker Inc., “Swarm mode overview,” <https://docs.docker.com/engine/swarm/>, 2024, Ultima accesare: Mai 2026.
- [8] HashiCorp, “Nomad documentation,” <https://developer.hashicorp.com/nomad/docs>, 2024, Ultima accesare: Mai 2026.

Anexe

Anexe prezintă doar elementele specifice proiectului!

Anexele (dacă este cazul) constituie o secțiune separată a lucrării care nu se numerează ca și capitol. Anexele se numerează crescător cu numere arabe (ex.: Anexa 1, Anexa 2 etc.). Anexele vor conține scheme, diagrame, secvențe din codul sursă.

Anexa 1. Componente Software

- diagramele UML care referă numai la componentele dezvoltate de student și care datorită complexității pot fi listate pe o foaie de tip A3 sau A2.
- cod sursă numai pentru componentele dezvoltate de către student.

Anexa 2. Componente Hardware

- schemele electrice finale realizate într-un CAD de profil;
- schemele cablajelor realizate pentru implementare, realizate într-un CAD de profil;
- informații suplimentare despre implementarea și testarea aplicației (de ex. capturi de osciloscop);
- schemele standard ce vor fi folosite pentru testare (pseudocod sau schemă logică);
- extrase din foi de catalog.

Anexa 3. Codul funcției xyz()

În Figura A.1 este prezentat unul dintre cele mai impunătoare vehicule ale Imperiului.

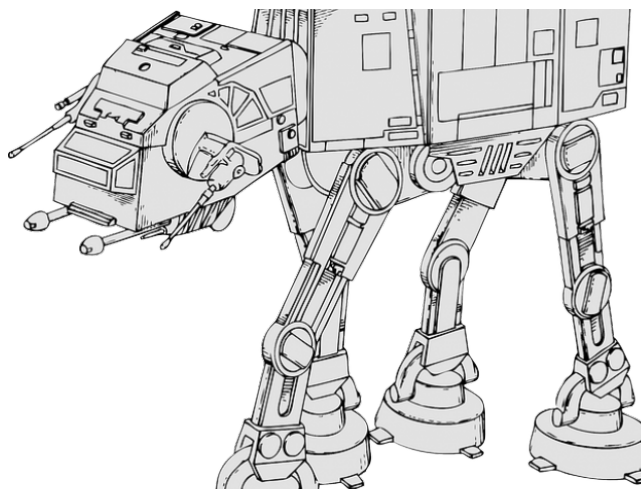


Figura A.1. AT²

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetur id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetur adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue

² imagine preluată de pe un site web care nu „merită” trecut la bibliografie <https://www.pngitem.com/>

quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetur.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus. (Figura A.2)

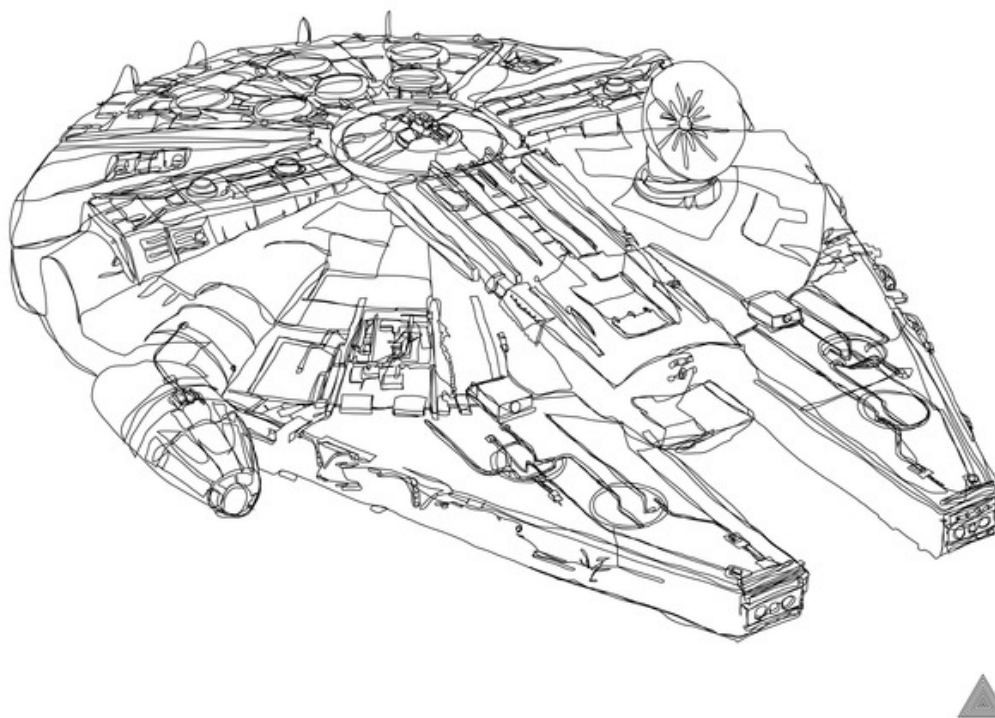


Figura A.2. Millennium Falcon³

³ imagine preluată de pe un site web care nu „merită” trecut la bibliografie <http://clipart-library.com/>

Anexa 4. Tabel lung pe mai multe pagini

Malware (software rău intenționat) este un termen folosit pentru a descrie orice program sau cod care ocolește procesele de control al accesului, este creat cu intenția de a ataca un sistem de calcul sau rețea, pentru a provoca daune sau a compromite acel sistem. În categoria malware se includ: viruși, *ransomware*, *keyloggers*, troieni, viermi, *spyware* etc (Tabelul A.1). În vocabularul curent, termenul de "virus" este folosit abuziv pentru orice tip de malware.

Tabelul A.1. Tipuri de malware

<i>Tip</i>	<i>Descriere</i>	<i>Exemple elocvente</i>
Virus	cod executabil malițios, atașat la fișiere sau programe legitime. Majoritatea virușilor necesită declanșarea de către utilizatorul final. Se răspândește prin intermediul unităților USB, discurilor optice, partajărilor în rețea sau e-mail-ului	Brain, Michelangelo, Melissa
Ransomware	conceput pentru a ține captiv un sistem informatic, de obicei prin criptarea datelor esențiale, dezactivează accesul victimei la date până la efectuarea unei plăți către atacator	WannaCry, Petya
Fileless Malware	face modificări la fișierele native ale sistemului de operare	Astaroth
Spyware	proiectat pentru a urmări acțiunile utilizatorului și a colecta date despre activitatea victimei, fără cunoștința acesteia	DarkHotel
Adware	livrează anunțuri în mod automat. Unele sunt însoțite de programe <i>spyware</i>	Fireball
Troian	execută operațiuni rău intenționate disimulate ca operațiuni dorite. Se deghizează în codul unui program cunoscut și se atașează de fișiere non-executabile	Emotet
Worms	se reproduce singur, se răspândește într-o rețea prin replicare, prin exploatarea vulnerabilităților din rețele. Au tipare similare, inclusiv o vulnerabilitate activă, o modalitate de a se propaga singuri și conțin un <i>payload</i>	Stuxnet
Rootkit	conceput pentru a modifica sistemul de operare, astfel încât computerul să poată fi accesat de la distanță, printr-o porțiță de acces. Rootkit-urile modifică privilegiile de acces, fișierele de sistem și instrumentele de monitorizare a sistemului, ceea ce le face foarte greu de detectat și eliminat	Zacinto
Keyloggers	monitorizează apăsările de taste ale victimei	Olympic Vision
Bots	lansează automat atacuri concertate și concentrate asupra sistemului țintă	Echobot

<i>Tip</i>	<i>Descriere</i>	<i>Exemple elocvente</i>
Mobile Malware	conțin cod specific (Android, iOS) infec- tării dispozitivele mobile	Triada

Anexa 5. Tabel simplu

Pentru a ușura lecturarea lucrării de diplomă recomandăm ca tabelele (dacă nu sunt prea mari) să nu fie împărțite (sparte) pe mai multe pagini.

Tabelul A.2. Rezultatele simulării

<i>Tipul semnalului</i>	<i>Durata</i>	<i>Randamentul</i>
sinusoidal	10s	99%
dreptunghiular	12s	81%
triunghiular	15s	89%

În Tabelul A.2 este prezentat un exemplu de formatare pentru un tabel cu cap de tabel orizontal.

Anexa 6. Cod în python

```
1  import numpy as np
2
3  def incmatrix(genl1,genl2):
4      m = len(genl1)
5      n = len(genl2)
6      M = None #to become the incidence matrix
7      VT = np.zeros((n*m,1), int) #dummy variable
8
9      #compute the bitwise xor matrix
10     M1 = bitxormatrix(genl1)
11     M2 = np.triu(bitxormatrix(genl2),1)
12
13     for i in range(m-1):
14         for j in range(i+1, m):
15             [r,c] = np.where(M2 == M1[i,j])
16             for k in range(len(r)):
17                 VT[(i)*n + r[k]] = 1;
18                 VT[(i)*n + c[k]] = 1;
19                 VT[(j)*n + r[k]] = 1;
20                 VT[(j)*n + c[k]] = 1;
21
22             if M is None:
23                 M = np.copy(VT)
24             else:
25                 M = np.concatenate((M, VT), 1)
26
27             VT = np.zeros((n*m,1), int)
28
29     return M
```

Listing 1. Codul funcției *incmatrix*

Anexa 7. Cod în Kotlin (mai lung de o pagină)

```

1  internal inner class ScheduleAdapter :
2      RecyclerView.Adapter<CalendarSessionViewHolder>(),
3      StickyHeaderHandler {
4
5      private var schedule: MutableList<ScheduleItem> = mutableListOf()
6      var data: List<SessionDataGroup> = emptyList()
7          set(value) {
8              updateSchedule(value)
9              field = value
10         }
11
12     override fun onCreateViewHolder(
13         parent: ViewGroup,
14         viewType: Int): CalendarSessionViewHolder {
15         val view = when (viewType) {
16             ScheduleItem.TYPE_SMALL ->
17                 R.layout.item_schedule_session_header_small
18             ScheduleItem.TYPE_LARGE ->
19                 R.layout.item_schedule_session_header_large
20             else -> R.layout.item_schedule_session_card
21         }
22
23         val holder = layoutInflater.inflate(view, parent, false)
24         return CalendarSessionViewHolder(holder, _favoriteNeedSync)
25     }
26     override fun getItemCount(): Int = schedule.size
27
28     override fun onBindViewHolder(
29         holder: CalendarSessionViewHolder, position: Int) {
30         holder.show(schedule[position])
31     }
32
33     override fun getItemViewType(position: Int): Int {
34         return schedule[position].type
35     }
36     private fun updateSchedule(groups: List<SessionDataGroup>) {
37         val result = mutableListOf<ScheduleItem>()
38         for (group in groups) {
39             if (group.type == 0) {
40                 result += ScheduleItem.SmallHeader(
41                     group.title, R.color.dark_grey_40
42                 )
43                 continue
44             }
45             if (group.type == 1) {
46                 result += ScheduleItem.LargeHeader(group)
47                 continue
48             }
49
50             result += ScheduleItem.LargeHeader(group)
51             result += group.sessions.map { ScheduleItem.Card(it) }
52         }
53     }

```

```
54         schedule = result
55     }
56     override fun getAdapterData(): MutableList<*> = schedule
57 }
```

Listing 2. *clasa ScheduleAdapter*

Anexa 8. Cod în XML (preluat din fisier)

```

1  <?xml version="1.0" encoding="UTF-8"?>
2  <project xmlns="http://maven.apache.org/POM/4.0.0"
3      xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
4      xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
5          https://maven.apache.org/xsd/maven-4.0.0.xsd">
6      <modelVersion>4.0.0</modelVersion>
7      <parent>
8          <groupId>org.springframework.boot</groupId>
9          <artifactId>spring-boot-starter-parent</artifactId>
10         <version>2.3.4.RELEASE</version>
11         <relativePath/> <!-- lookup parent from repository -->
12     </parent>
13     <groupId>local.pos.examples</groupId>
14     <artifactId>CalculatorWS</artifactId>
15     <version>0.0.1-SNAPSHOT</version>
16     <name>CalculatorWS</name>
17     <description>Simple WS: Stateless Calculator</description>
18
19     <properties>
20         <java.version>11</java.version>
21     </properties>
22
23     <dependencies>
24         <dependency>
25             <groupId>org.springframework.boot</groupId>
26             <artifactId>spring-boot-starter-web</artifactId>
27         </dependency>
28         <dependency>
29             <groupId>org.springframework.boot</groupId>
30             <artifactId>spring-boot-starter-web-services</artifactId>
31         </dependency>
32
33         <dependency>
34             <groupId>org.springframework.boot</groupId>
35             <artifactId>spring-boot-starter-test</artifactId>
36             <scope>test</scope>
37             <exclusions>
38                 <exclusion>
39                     <groupId>org.junit.vintage</groupId>
40                     <artifactId>junit-vintage-engine</artifactId>
41                 </exclusion>
42             </exclusions>
43         </dependency>
44
45         <dependency>
46             <groupId>wsdl4j</groupId>
47             <artifactId>wsdl4j</artifactId>
48         </dependency>
49
50         <dependency>
51             <groupId>javax.xml.bind</groupId>
52             <artifactId>jaxb-api</artifactId>
53         </dependency>

```

```
54
55     <dependency>
56         <groupId>javax.activation</groupId>
57         <artifactId>activation</artifactId>
58         <version>1.1</version>
59     </dependency>
60
61     <dependency>
62         <groupId>org.glassfish.jaxb</groupId>
63         <artifactId>jaxb-runtime</artifactId>
64     </dependency>
65 </dependencies>
66
67 <build>
68     <plugins>
69         <plugin>
70             <groupId>org.springframework.boot</groupId>
71             <artifactId>spring-boot-maven-plugin</artifactId>
72         </plugin>
73
74         <plugin>
75             <groupId>org.codehaus.mojo</groupId>
76             <artifactId>jaxb2-maven-plugin</artifactId>
77             <version>2.5.0</version>
78             <executions>
79                 <execution>
80                     <id>xjc</id>
81                     <goals>
82                         <goal>xjc</goal>
83                     </goals>
84                 </execution>
85             </executions>
86             <configuration>
87                 <sources>
88                     <source>
89 ${project.basedir}/src/main/resources/Calculator.xsd
90                     </source>
91                 </sources>
92             </configuration>
93         </plugin>
94     </plugins>
95 </build>
96
97 </project>
```

Listing 3. Exemplu de cod XML preluat din fișier extern